

Kubernetes: Networking

VISHWANATH M S
VISHWACLOUDLAB.IN



Agenda

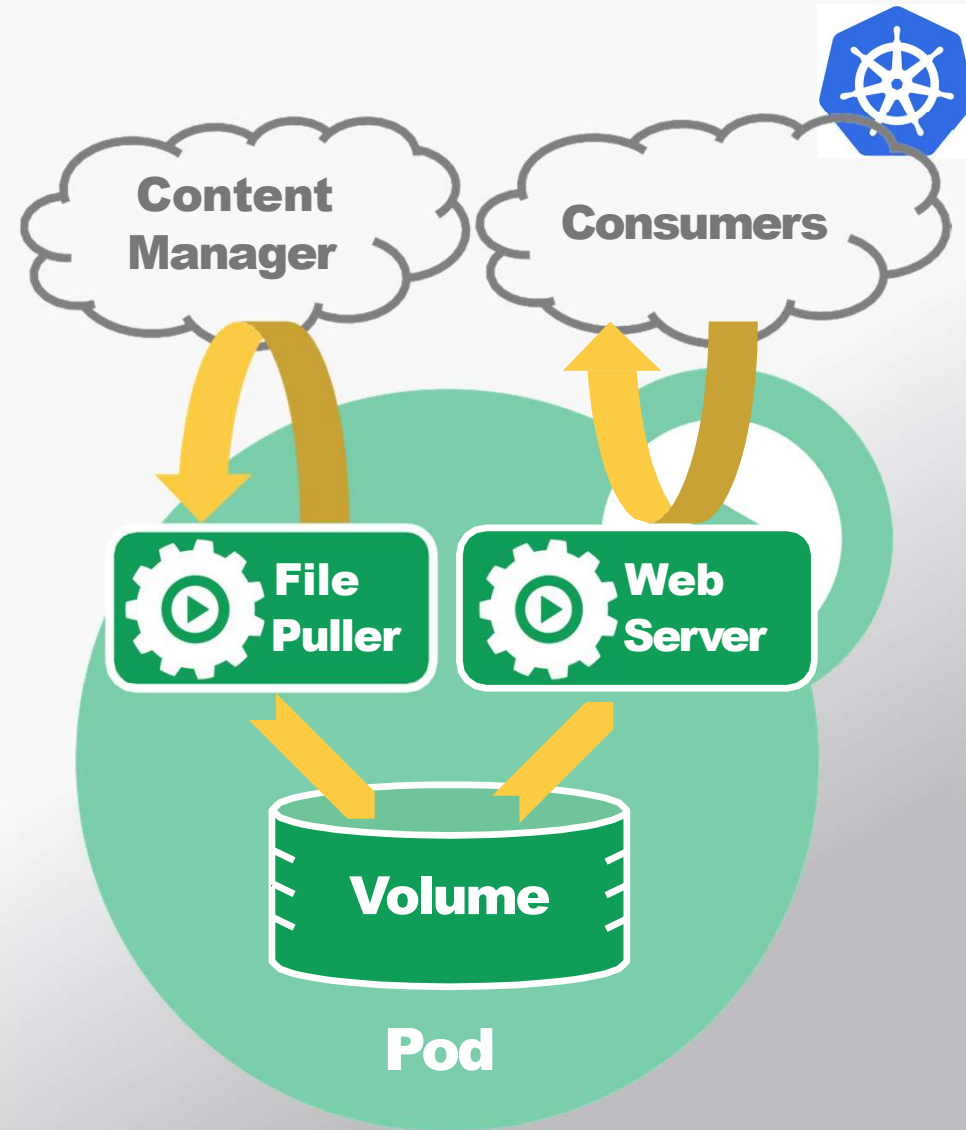
- Single pod networking
- Pod to pod Networking(East-West traffic)
- Service discovery and load balancing
- External access(North-South traffic)
- Network policy
- Istio service mesh
- Multi-cluster and hybrid cloud
- Best practises

Pod

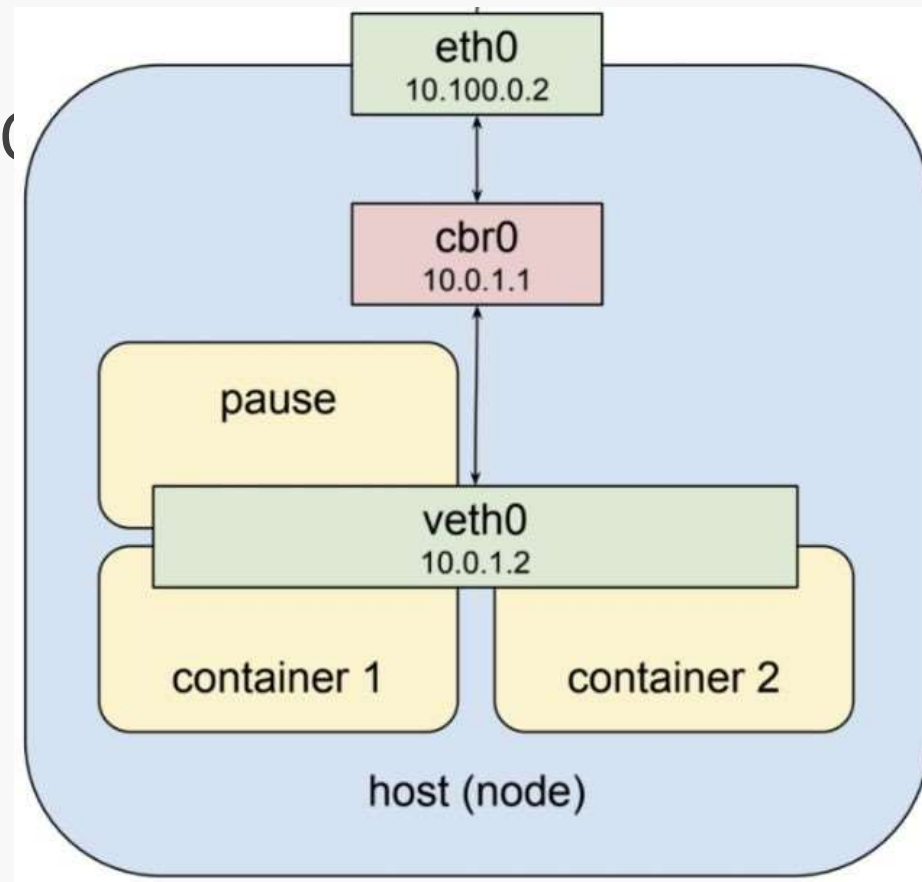
S

- Small group of tightly-coupled containers
- Every pod has a unique IP
- Share same namespace and volume
- Share same lifecycle
- Use cases for multi-container pods -
Sidecars, proxies/adapters

Example: File puller & web server



Single Po



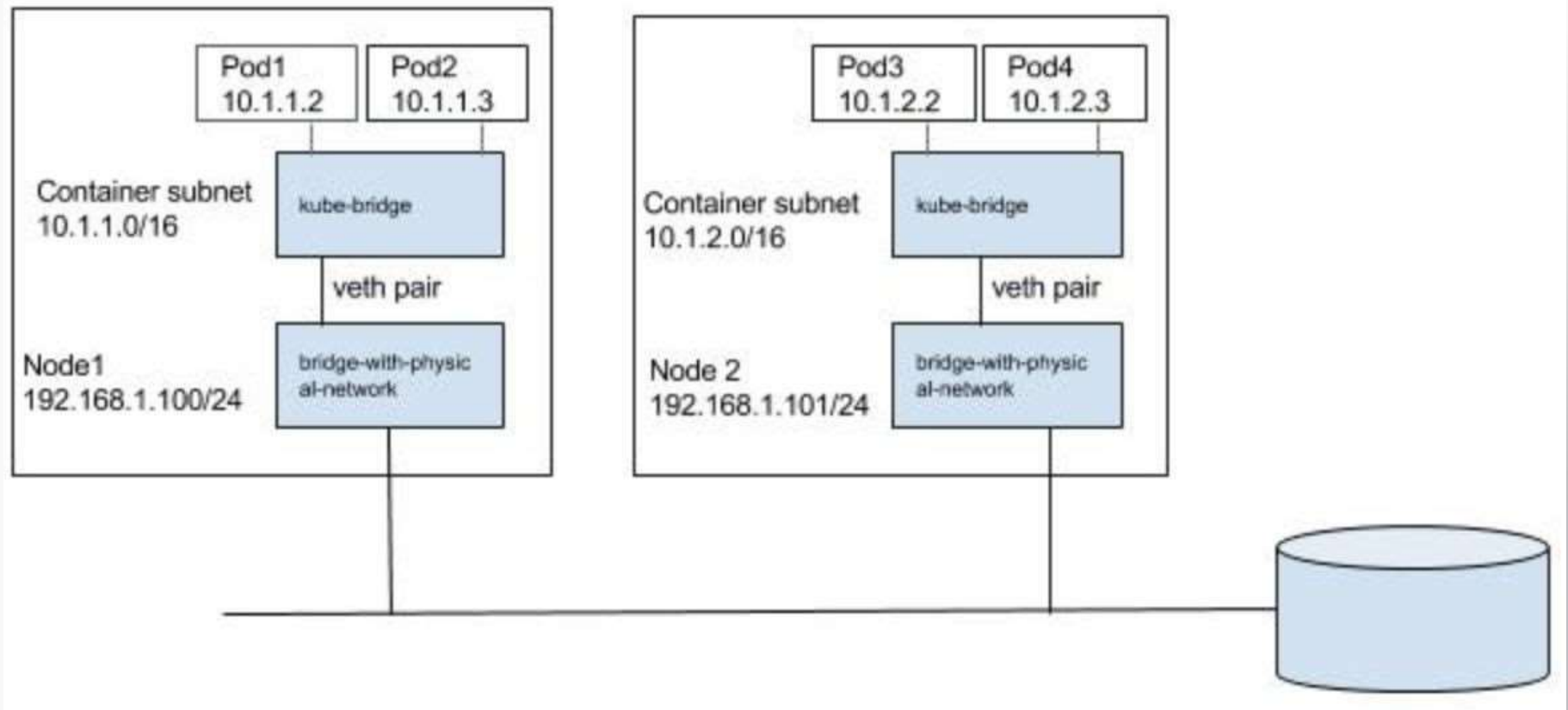
nodes

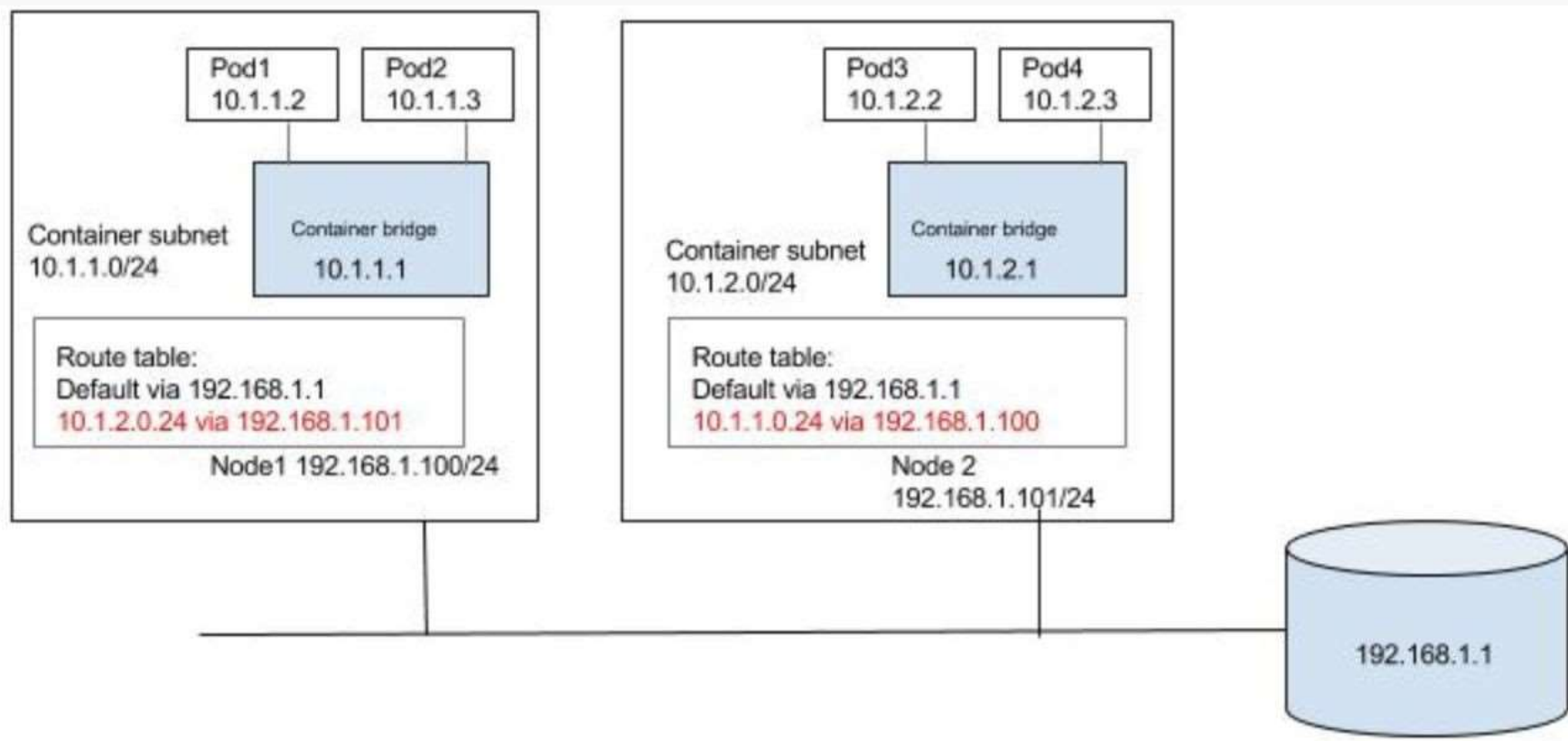


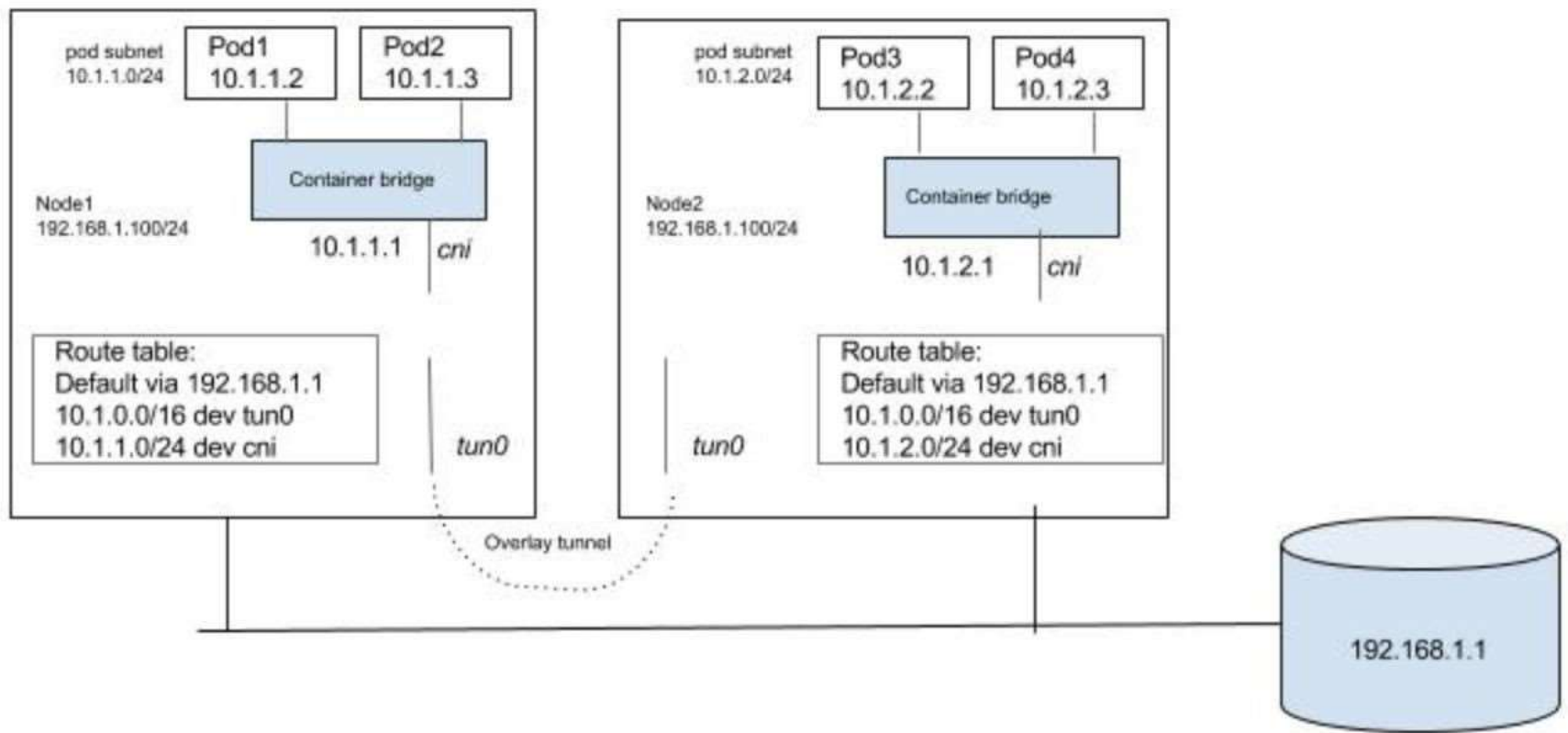
Kubernetes supports different networking approaches to do pod networking

Plugins can be added using CNI

Type/Features	L2	L3	Overlay	Cloud
Summary	Pods communicate using L2	Pod traffic is routed in underlay network	Pod traffic is encapsulated and uses underlay for reachability	Pod traffic is routed in cloud virtual network
Underlying technology	L2 arp, broadcast	Routing protocol like BGP	VXLAN	Pre-programmed fabric using controller(eg: GKE)
Encapsulation	No double encapsulation	No double encapsulation	Double encapsulation	No double encapsulation
Examples	KUBERNETES	Calico	Flannel, Weave	GKE, ACS, EKS





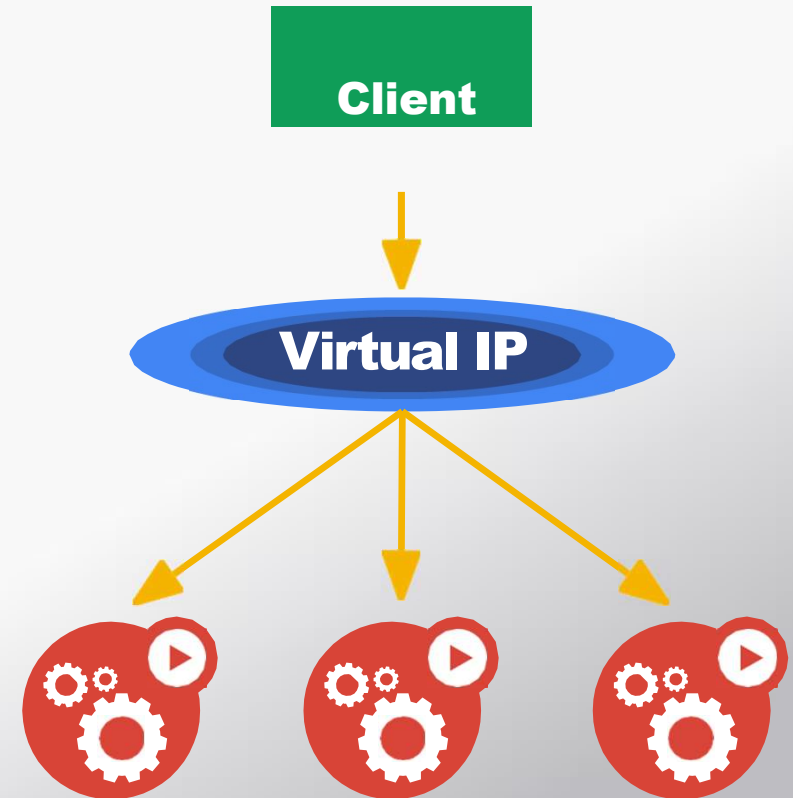




Services

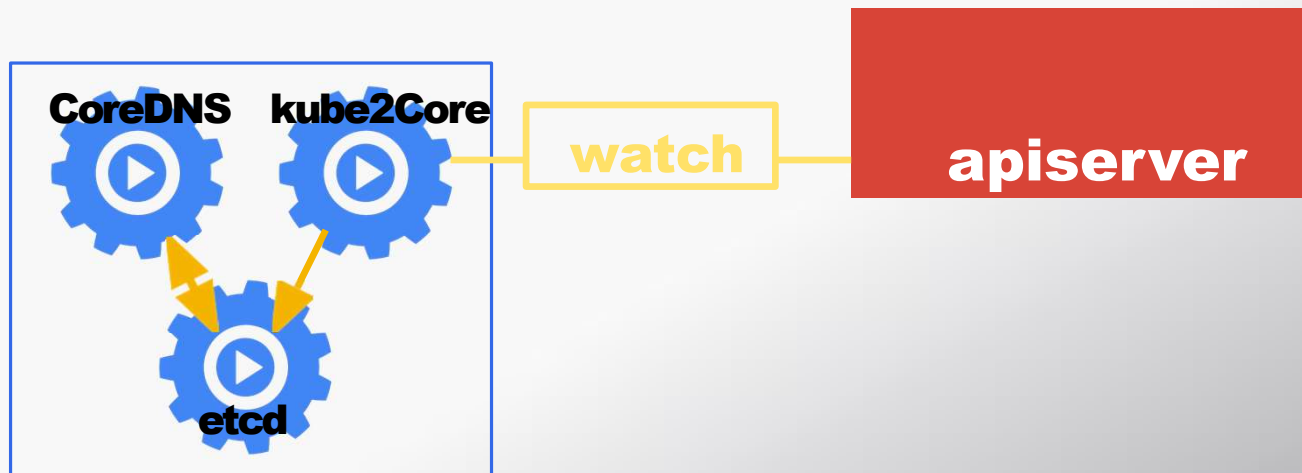
- A service is a group of endpoints (usually pods) grouped by selector
- Services provide a stable VIP (except Headless)
- VIP automatically routes to backend pods
- VIP to backend pod mapping is managed by kube-proxy, implemented using iptables

```
apiVersion: v1
kind: Service
metadata:
  name: productpage
spec:
  selector:
    app: productpage
  type: ClusterIP
  ports:
    - port: 80
      targetPort: 8080
      protocol: TCP
```



DNS

- Service Discovery achieved by CoreDNS
- CoreDNS runs as a pod in the cluster





Service communication

- Service IP is accessible only from within the cluster
- Type “ClusterIP” is used for internal communication within the cluster
- Traffic sent to Service IP gets load balanced to pods that belong to service IP
- Source NAT is used for pods to communicate to external world
- IPtables are used extensively for load balancing and NAT
- Nodeport, Network load balancer and Ingress controller are service types for external world to reach services inside the cluster.

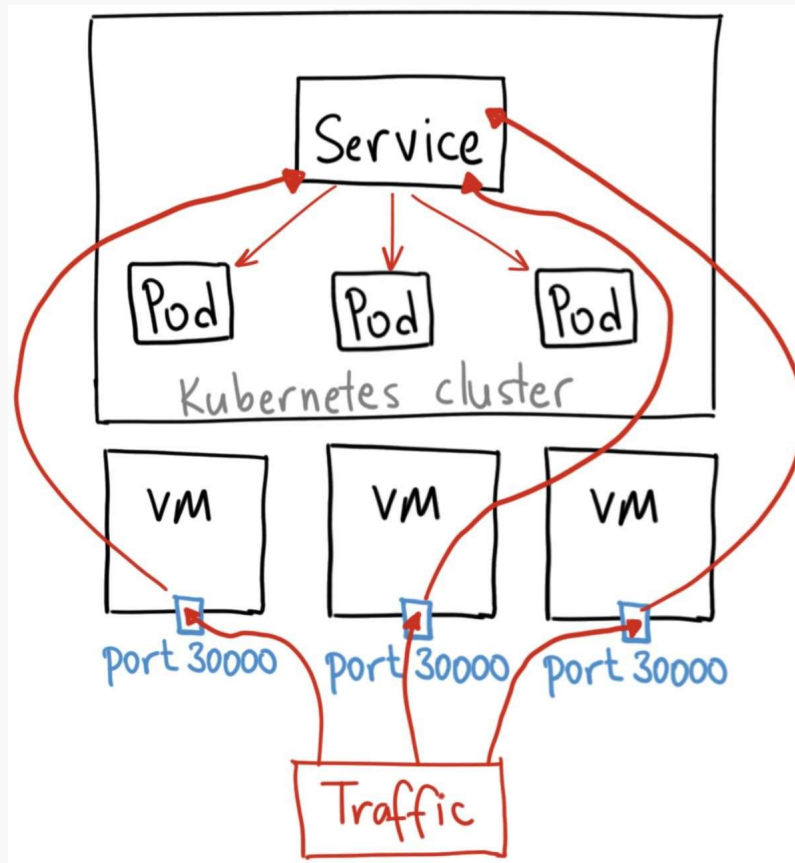


Service external reachability options

Feature	Nodeport	Load balancer	Ingress
Summary	Service is exposed using a reserved port in all nodes of cluster(Default: 32000-32767)	Typically implemented as network load balancer	Typically implemented as http load balancer
IP address	Node IP is used for external communication	Each service needs to have own external IP	Many services can share same external IP, uses path based demux
Use Case	Demo purposes	L3 services	L7 services
Examples		GKE Network load balancer	GKE http load balancer, nginx, Istio



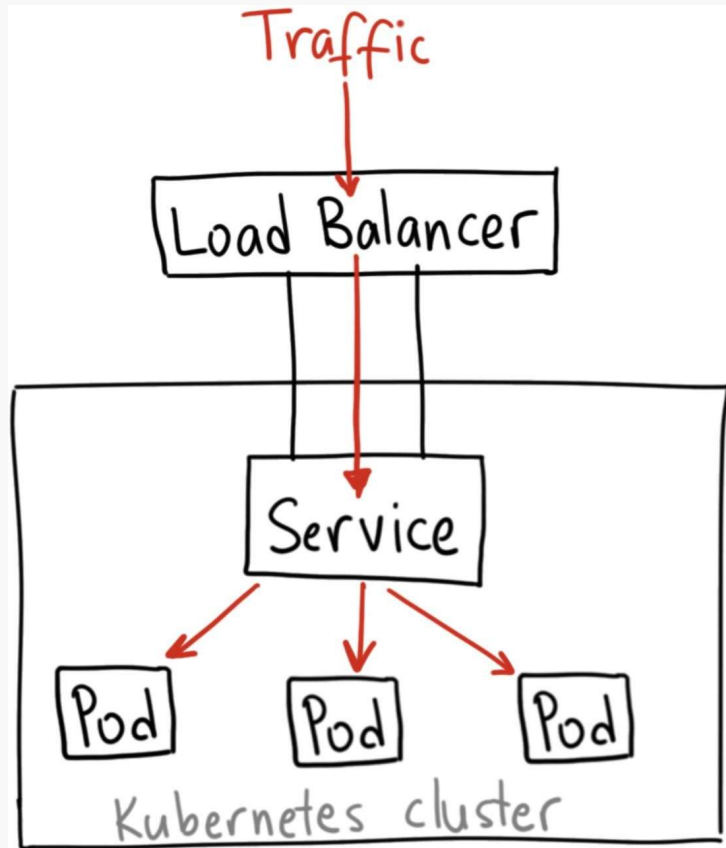
Nodeport



```
apiVersion: v1
kind: Service
metadata:
  name: productpage
  labels:
    app: productpage
spec:
  type: NodePort
  ports:
    - port: 30000
      targetPort: 9080
  selector:
    app: productpage
```



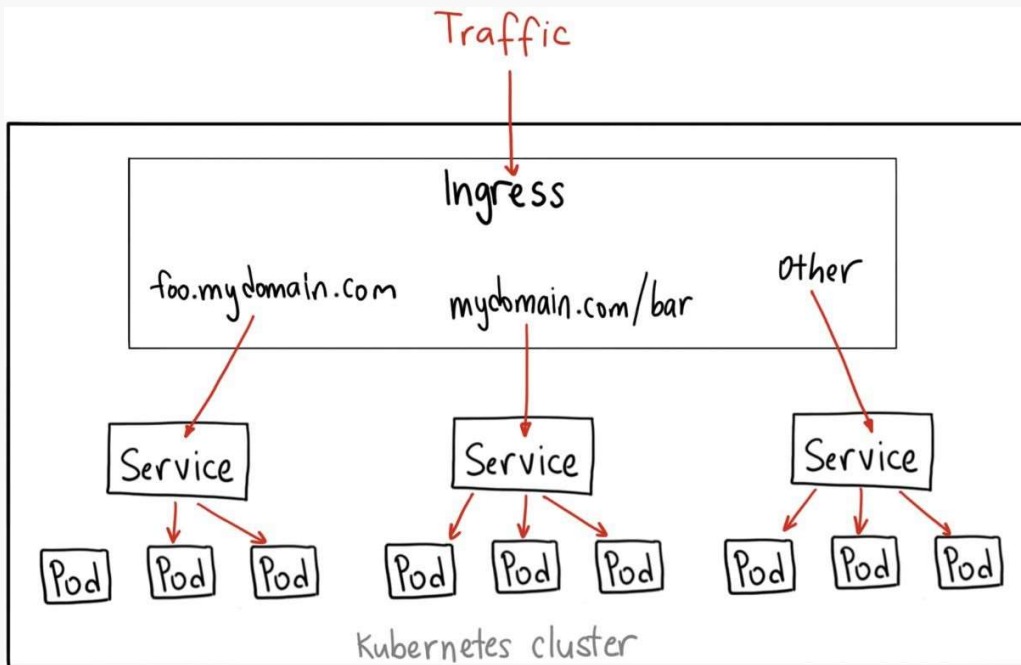
Load balancer



```
apiVersion: v1
kind: Service
metadata:
  name: productpage
  labels:
    app: productpage
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 9080
  selector:
    app: productpage
```



Ingress



apiVersion: extensions/v1beta1

*kind: **Ingress***

metadata:

name: gateway

spec:

backend:

serviceName: productpage

servicePort: 9080

rules:

- host: mydomain.com

http:

paths:

- path: /productpage

backend:

serviceName: productpage

servicePort: 9080

- path: /login

backend:

serviceName: productpage

servicePort: 9080

*- path: /**

backend:

serviceName: productpage

servicePort: 9080



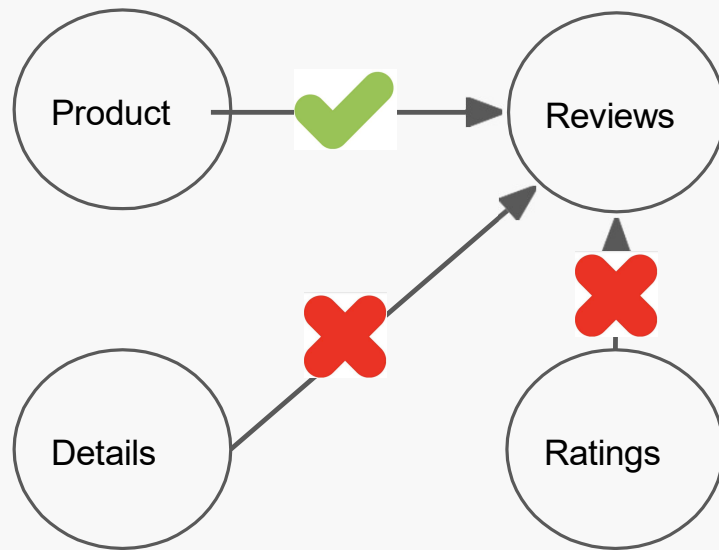
Network control policy

Controls communication between Pods and Services

- Traffic is by default allowed without network policy
- Kubernetes network policies are allow policies, there is no deny policy
- Empty pod selector selects all pods
- Ingress and egress rules are available to control traffic
- Traffic is allowed if atleast 1 policy allows traffic
- Network plugins like Calico, Weavenet support network policies. GKE uses calico



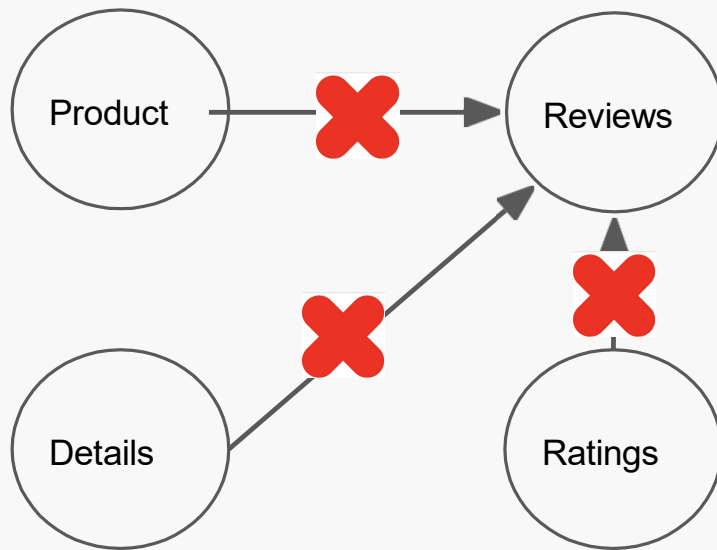
Network control policy example1



```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: hello-allow-from-product
spec:
  policyTypes:
  - Ingress
  podSelector:
    matchLabels:
      app: reviews
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: productpage
```

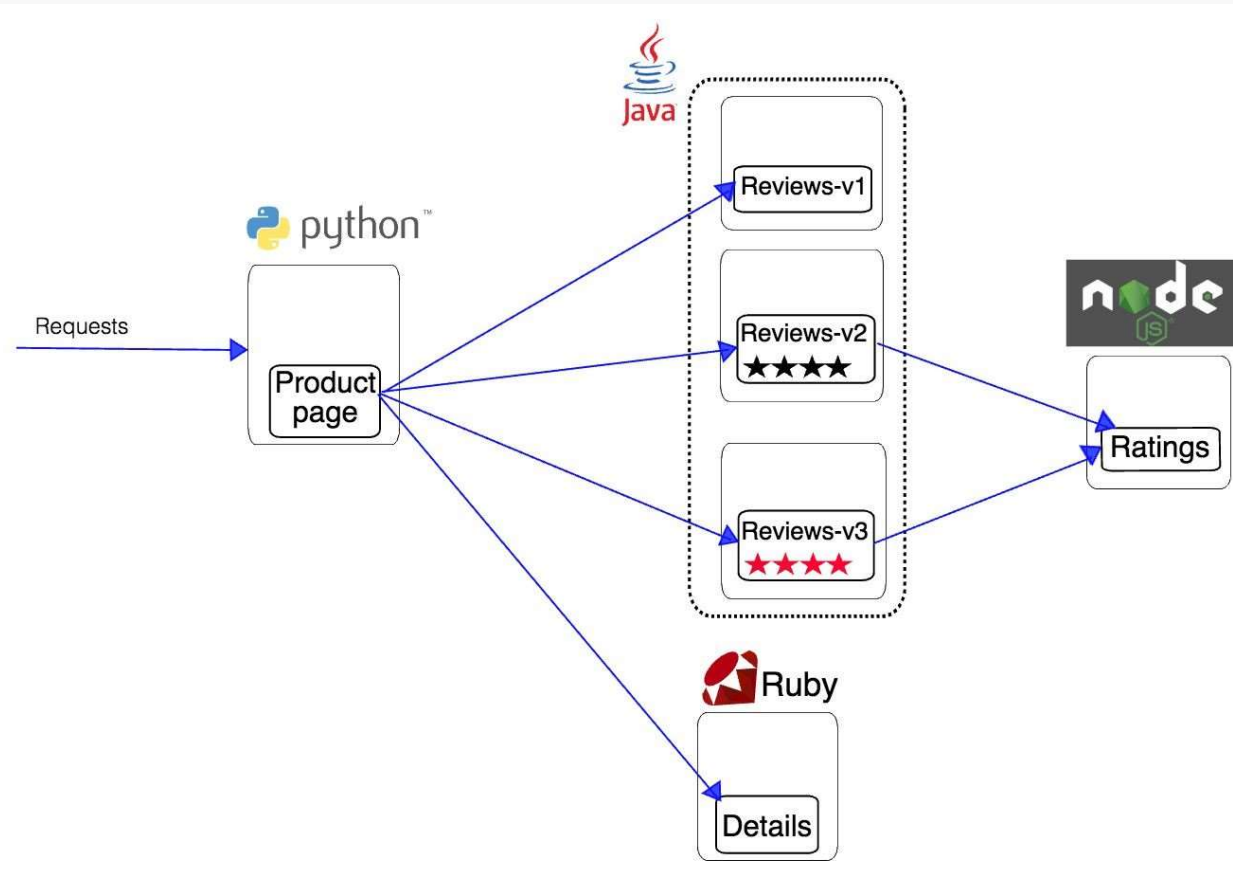


Network control policy example2



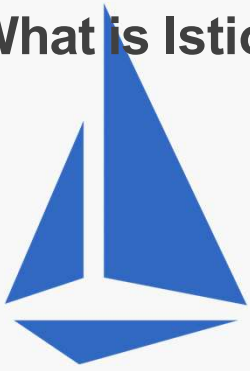
```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: hello-allow-from-product
spec:
  policyTypes:
    - Ingress
  podSelector:
    matchLabels:
      app: reviews
  ingress: []
```

Book





What is Istio?



Istio is an open framework for **connecting, securing, managing** and **monitoring** services

What can Istio do?



Traffic control



Observability



Security



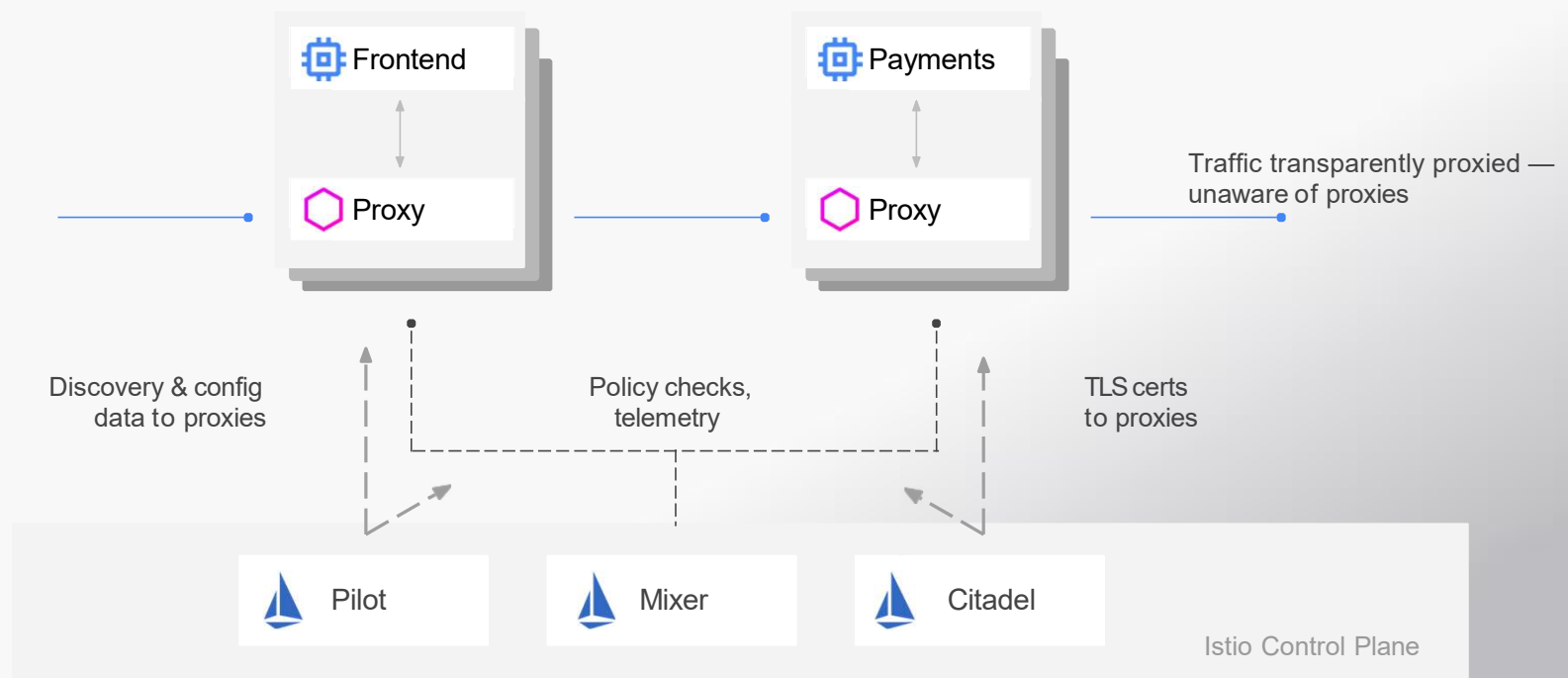
Fault-injection



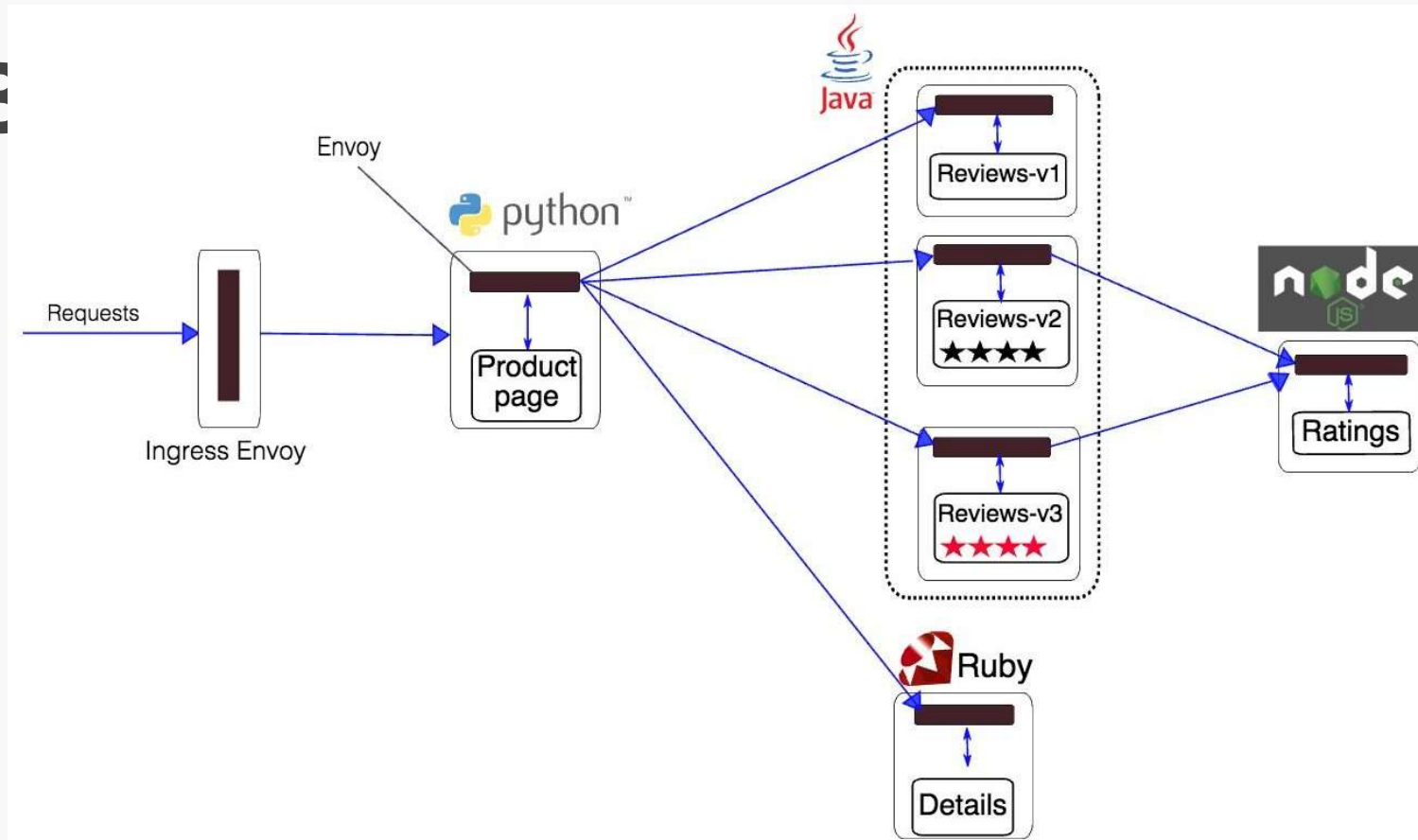
Hybrid cloud



Istio Architecture



E



Compare Network control policy with Istio

	Istio Policy	Network Policy
Layer	“Service” – L7	“Network” – L3-4
Implementation	User space	Kernel
Enforcement Point	Pod	Node



Multi-cluster

Use- cases:

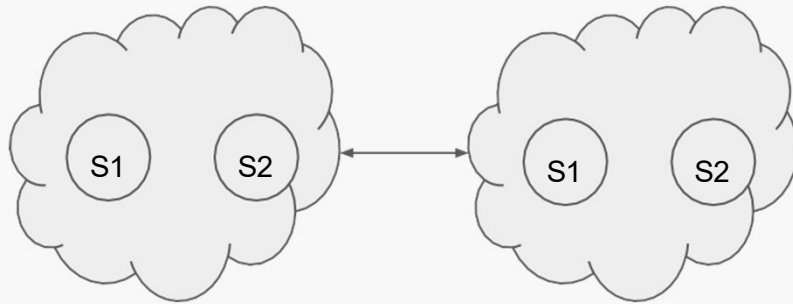
- 1 cluster serving as DR for another cluster
- CI/CD between Dev and production clusters
- Global load balancing across multiple clusters
- Ability for 1 cluster to consume services from another cluster

Requirements:

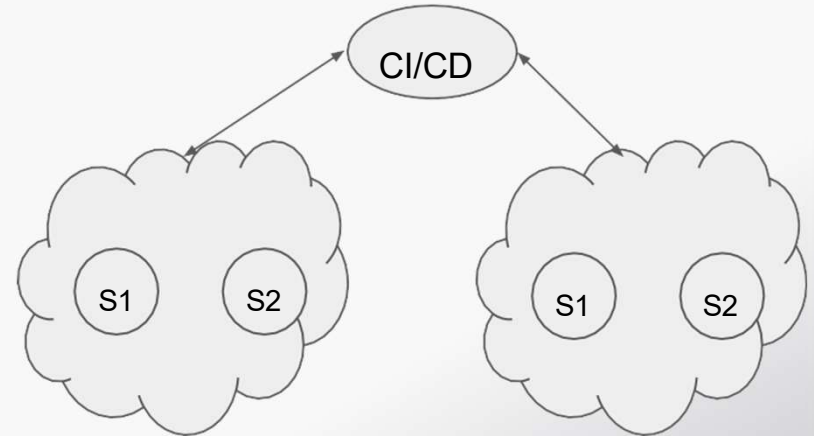
- Common control plane across clusters
- Service discovery across clusters
- Load balancing traffic across replicas in different clusters
- Common policy management across clusters - RBAC, Quota etc

Multi-Cloud, Hybrid Cloud use

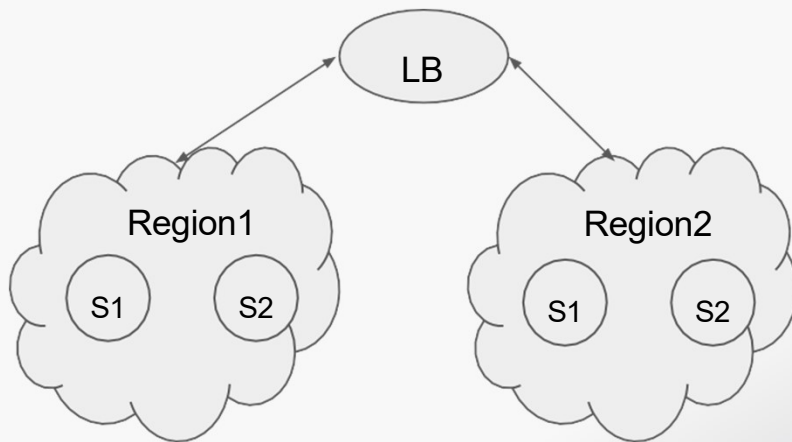
cases



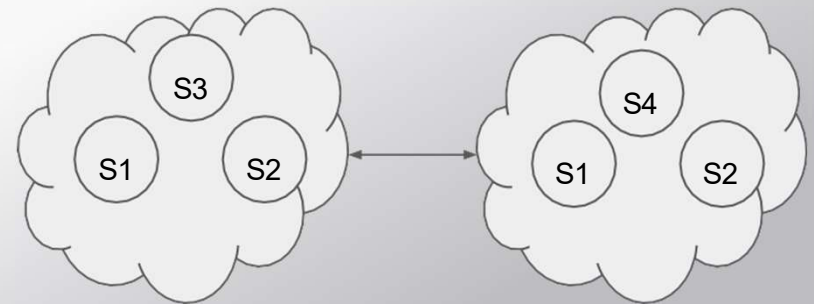
Disaster Recovery



Dev/Prod setup



Load balance across regions



Cloud bursting
Consume services across cluster



Multi-cluster deployment options

- **Kubernetes Federated cluster**
 - Have a single Kubernetes control plane that spans multiple clusters
 - Controlled using Kubefed
 - Service discovery works across clusters
- **Istio Multicluster**
 - Istio control plane that spans across multiple clusters
 - Kubernetes control plane limited to single cluster
 - Service discovery works across clusters
 - Istio ingress takes care of load balancing external traffic across clusters
- **Managed service from cloud providers (eg: GCP Cloud services platform)**

CDK Hybrid cloud Kubernetes solution





Kubernetes Networking - Best practises

- Use namespaces for multi-tenancy
- Use Ingress rather than Load balancer to expose services. This is to preserve resources
- Do not expose cluster to outside world if its not needed
- Create security policies like IAM, RBAC, Network control in the beginning rather than later
- Put emphasis on resource control including quotas, priorities/preemption etc

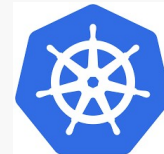
Our Agenda



- 01 RBAC in Kubernetes
- 02 Why RBAC ?
- 03 Benefits of using RBAC
- 04 Implementing RBAC
- 05 Demo

RBAC in our daily life

Roles of different individuals



Each individual in today's world, is assigned a specific role and is expected to work in the allotted field, for example like HR, Finance, Engineering, Sales etc. without hampering the work of any other individual belonging to any other domain.

This brings out the need for some access control policy, in order to restrict the permissions allotted to a user.

